

Với tư cách là Software Architect, bạn sẽ cùng tôi thiết kế một hệ thống quản lý tài liệu thế hệ mới (tạm gọi là **Ebook manager**). Trọng tâm của hệ thống này là "**Tối đa hóa Siêu dữ liệu (Metadata)**" và "**Phân loại Động (Dynamic Classification)**" thay vì nhốt file vào các thư mục cứng nhắc.

Dưới đây là bản phân tích ý tưởng lớn và Thiết kế Kiến trúc tổng thể cho hệ thống này.

1. Phương pháp tổ chức khoa học (Scientific Organization Method)

Thay vì dùng cây thư mục (Folders) truyền thống, chương trình sẽ quản lý theo phương pháp **Faceted Classification (Phân loại đa khía cạnh)** kết hợp **Smart Tags**.

- **Single Source of Truth (Nguồn chân lý duy nhất):** Phần mềm KHÔNG copy file của bạn đi nơi khác như Calibre. Nó chỉ quét (Scan) các thư mục bạn chỉ định trên ổ cứng và tạo Chỉ mục (Index). File của bạn vẫn nằm nguyên ở chỗ cũ.
- **Virtual Collections (Bộ sưu tập ảo):** Dựa trên Metadata, bạn có thể tạo các rule (quy tắc). Ví dụ: Một Collection tự động hiển thị "*Tất cả file PDF, có từ khóa 'Python', xuất bản sau năm 2023*".

2. Chiến lược khai thác tối đa thuộc tính (Property Extraction)

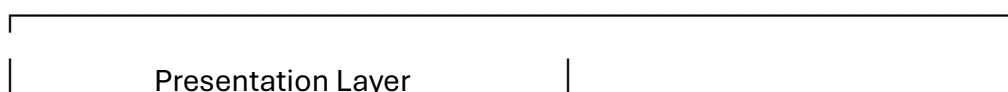
Hệ thống sẽ có các "cảm biến" (Extractors) chuyên dụng cho từng loại file để hút kiệt dữ liệu:

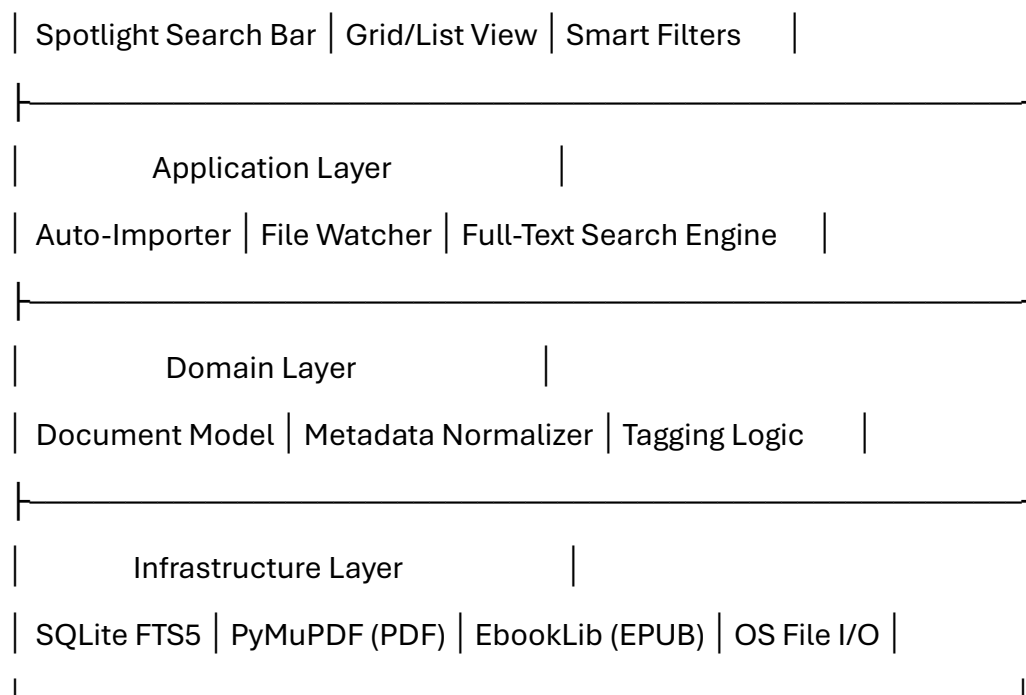
- **File PDF:** Trích xuất Tiêu đề, Tác giả, Ngày tạo file, Keywords nhúng trong file, Mục lục (Table of Contents - cực kỳ quan trọng để tìm kiếm nội dung), và các đoạn Highlight/Ghi chú (Annotations) bạn đã đánh dấu.
- **File EPUB / AZW3 / MOBI:** Giải mã file nén, đọc file content.opf (chuẩn Dublin Core) để lấy chính xác Tên sách, Tác giả, Nhà xuất bản, ISBN, Ngôn ngữ, và trích xuất trực tiếp Ảnh bìa gốc (Cover) mà không làm biến đổi file.
- **Hệ điều hành (OS Properties):** Khai thác Date Created, Date Modified, File Size, File Path.

3. Kiến trúc phần mềm đề xuất (Clean Architecture)

Để đảm bảo chương trình chạy siêu tốc, tìm kiếm nội dung trong chớp mắt và dễ dàng mở rộng, chúng ta sẽ thiết kế theo kiến trúc sau:

Plaintext





Các Module cốt lõi (Core Modules) cần có:

1. **Metadata Extractor Engine:** Bộ máy dùng PyMuPDF (cho PDF) và EbookLib (cho EPUB) để bóc tách thuộc tính mà không cần mở file lên giao diện.
2. **SQLite FTS5 Database:** Chìa khóa cho tốc độ. Thay vì dùng Database SQL thông thường, chúng ta sẽ bật module FTS5 (Full-Text Search) của SQLite. Nó sẽ lập chỉ mục toàn bộ nội dung (text) của hàng ngàn cuốn sách, cho phép bạn gõ một câu văn và tìm ra cuốn sách chứa câu đó chỉ trong vài mili-giây.
3. **File Watcher Service:** Một tiến trình chạy ngầm (Background Worker) liên tục theo dõi các thư mục tài liệu trên máy tính. Cứ có file mới được tải về, nó tự động hút Metadata và Index nội dung ngay lập tức.
4. **Omnibar UI (Giao diện Tìm kiếm đa năng):** Một thanh tìm kiếm tối giản ở giữa màn hình. Gõ author:nam để tìm tác giả, hoặc gõ một đoạn văn để tìm nội dung sách.

Bước tiếp theo để thực thi

DANH SÁCH TDD CHO DỰ ÁN SMARTDOC LIBRARY

Dự án được tối ưu hóa thành 15 module lõi tập trung vào quản lý Metadata và tốc độ tìm kiếm:

- TDD-001: SQLite FTS5 Search Database (Trái tim của hệ thống)

- TDD-002: Document Models & Normalizer (Cấu trúc dữ liệu chuẩn hóa)
- TDD-003: EPUB & AZW3 Extractor (Trích xuất sách điện tử)
- TDD-004: PDF Extractor (Trích xuất PDF & Mục lục)
- TDD-005: Cover Art & Cache Manager (Quản lý ảnh bìa siêu tốc)
- TDD-006: File System Watcher (Theo dõi thư mục tự động)
- TDD-007: Background Job Queue (Hàng đợi xử lý file nặng)
- TDD-008: Event Bus System (Giao tiếp module)
- TDD-009: Smart Rules & Virtual Collections (Bộ lọc thông minh)
- TDD-010: Main Window & UI Shell (Khung giao diện chính)
- TDD-011: Omnibar Search UI (Thanh tìm kiếm siêu tốc)
- TDD-012: Faceted Filter Panel (Bộ lọc đa khía cạnh bên trái)
- TDD-013: Grid & List View (Hiển thị sách)
- TDD-014: File Action Engine (Mở file, Mở thư mục, Xóa)
- TDD-015: App Configuration (Cài đặt cấu hình)
- TDD-016: Cloud Review System (Google Drive Sync)
- TDD-017: AI Book Review & Preview Engine
- **TDD-018: Semantic Search Engine (Tìm kiếm ngữ nghĩa)**
- TDD-019: Personal Cloud Exporter (Send to Cloud)

Bối cảnh cho Agent 2:

Tôi là Tech Lead của dự án SmartDoc Library (phần mềm quản lý sách/tài liệu tối ưu hóa Metadata, code bằng Python). Hãy đọc kỹ tài liệu thiết kế kỹ thuật **TDD-001** dưới đây và thực thi việc viết code.

TDD-001: SQLite FTS5 Search Database

1. Mục tiêu:

Xây dựng Database cục bộ sử dụng module FTS5 (Full-Text Search) của SQLite. Hệ thống này cho phép lập chỉ mục (index) siêu dữ liệu và nội dung của hàng vạn file tài liệu, đảm bảo tốc độ truy vấn từ khóa chỉ trong vài mili-giây.

2. Phạm vi:

- Tạo file database cục bộ (ví dụ: library.db).
- Khởi tạo các bảng lưu trữ Metadata chuẩn và Bảng ảo (Virtual Table) FTS5.
- Cung cấp cơ chế tìm kiếm Full-text hỗ trợ cú pháp nâng cao (như tìm theo tiền tố *, tìm chính xác cụm từ).

3. Trách nhiệm của module:

- Cung cấp API nội bộ cho phép Insert/Update/Delete tài liệu.
- Cung cấp API search(query_string) trả về danh sách ID của tài liệu khớp kết quả.
- Đảm bảo an toàn khi đọc/ghi đa luồng (Thread-safe) vì background worker sẽ liên tục đẩy dữ liệu vào DB.

4. Kiến trúc:

- Thuộc tầng: Infrastructure Layer.
- Các module như UI (Omnibar Search) và Background Importer sẽ gọi vào đây.

5. Sơ đồ UML/PlantUML:

Đoạn mã

@startuml

skinparam componentStyle rectangle

component "Search UI" as ui

component "File Importer" as imp

component "DatabaseManager" as db

database "SQLite FTS5" as sqlite

imp -> db : insert_document(metadata)

ui -> db : search_fts("author:nam python")

db -> sqlite : SELECT ... MATCH ...

sqlite -> db : result rows

db -> ui : List[dict]

@enduml

6. Class Diagram:

Đoạn mã

@startuml

```
class DatabaseManager {  
    - db_path: str  
    - connection: sqlite3.Connection  
    - write_lock: threading.Lock  
    + connect() : void  
    + initialize_tables() : void  
    + insert_document(doc_id: str, title: str, author: str, path: str, tags: str, content: str) : void  
    + search_documents(keyword: str) : List[dict]  
}
```

@enduml

7. Schema thiết kế cơ sở dữ liệu:

Bắt buộc phải có 2 bảng liên kết với nhau bằng Triggers để tự động đồng bộ FTS:

- Bảng chính documents: id (PK, text), title (text), author (text), file_path (text), file_size (int), tags (text), created_at (real).
- Bảng ảo FTS5 documents_fts: Sử dụng câu lệnh CREATE VIRTUAL TABLE documents_fts USING fts5(title, author, tags, content, content='documents', content_rowid='id');

8. Interface / Abstract Base Classes: Không yêu cầu.

9. API nội bộ (Public Methods):

- initialize_tables()
- add_or_update_document(doc_id, metadata_dict, extracted_text)
- search(query_string) -> List[dict]
- delete_document(doc_id)

10. Luồng xử lý (Data Flow):

- **Thêm file:** Background Importer nạp file -> Gọi `add_or_update_document` -> DB acquire Lock -> Ghi vào bảng `documents` -> Trigger tự động đẩy `title`, `author`, `tags` và bản text nội dung vào `documents_fts` -> Release Lock.
- **Tìm kiếm:** Gõ chữ -> UI gọi `search("data")` -> DB thực thi `SELECT * FROM documents WHERE id IN (SELECT rowid FROM documents_fts WHERE documents_fts MATCH 'data*')` -> Trả về danh sách.

11. Threading model:

- Tương tự TDD-006 của PA Studio, phải bật `check_same_thread=False` và `PRAGMA journal_mode=WAL`; để đọc ghi không bị chặn. Cần 1 `threading.Lock()` cho các thao tác write.

12. Xử lý lỗi:

- Bắt lỗi `sqlite3.OperationalError` khi file DB bị khóa bởi một tiến trình bên ngoài.

13. Logging:

- Báo lỗi ERROR nếu câu lệnh FTS MATCH bị sai cú pháp do người dùng gõ ký tự đặc biệt.

14. Unit Test:

- Test khởi tạo database trong RAM (`:memory:`).
- Thêm 3 document (có metadata chứa chữ 'python' và 'java').
- Test gọi hàm `search('py*')` và assert trả về đúng 1 document.

15. Tiêu chuẩn coding:

- Code phải sanitize (làm sạch) đầu vào của hàm `search` để tránh FTS Syntax Error (ví dụ xử lý các dấu ngoặc kép, ký tự lạ trước khi truyền vào MATCH).

16. Checklist nghiệm thu:

- [] Bảng FTS5 được tạo thành công.
- [] Hỗ trợ đa luồng (WAL mode).
- [] Tìm kiếm trả về kết quả dưới 50ms.

17. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-001, viết module database_manager.py cho dự án SmartDoc Library. Sử dụng thư viện sqlite3. Bật PRAGMA WAL. Xây dựng class DatabaseManager. Viết hàm initialize_tables tạo bảng documents và tạo bảng ảo CREATE VIRTUAL TABLE documents_fts USING fts5(title, author, tags, content). Cần tự viết các TRIGGER (AFTER INSERT, AFTER DELETE, AFTER UPDATE) để bảng FTS5 tự động đồng bộ dữ liệu từ bảng documents. Viết hàm add_document và hàm search(keyword). Trong hàm search, tự động thêm dấu * vào sau keyword để hỗ trợ tìm kiếm theo tiền tố. Chặn lỗi FTS bằng cách thay thế các ký tự đặc biệt trong keyword. Viết code test ở cuối file dùng :memory:, chèn vài sách ảo và gọi search."

TDD-002: Document Models & Normalizer

1. Mục tiêu:

Định nghĩa cấu trúc dữ liệu cốt lõi (Data Models) cho một tài liệu/sách và cung cấp bộ chuẩn hóa (Normalizer) để làm sạch siêu dữ liệu (Metadata) rác thường gặp trong các file tải trên mạng.

2. Phạm vi:

- Định nghĩa class Document chứa đầy đủ thuộc tính (Title, Author, Path, Ext, Size, Tags).
- Làm sạch chuỗi văn bản: Xóa khoảng trắng thừa, chuẩn hóa tên tác giả (VD: "Nam, Nguyen" thành "Nguyen Nam"), loại bỏ các ký tự đặc biệt không hợp lệ.

3. Trách nhiệm của module:

- Cung cấp Schema chung để tất cả các module khác (Database, UI, Extractor) giao tiếp với nhau.
- Tự động gán UUID duy nhất cho mỗi tài liệu dựa trên mã băm (Hash) của đường dẫn file hoặc nội dung file.

4. Kiến trúc:

- Thuộc tầng: Domain Layer.

5. Sơ đồ UML/PlantUML:

Đoạn mã

@startuml

skinparam componentStyle rectangle

```
component "PDF Extractor" as pdf
component "Normalizer" as norm
component "Document Model" as doc
component "Database" as db
```

```
pdf -- norm
```

```
norm -- doc
```

```
doc -- db
```

```
@enduml
```

6. Class Diagram:

Đoạn mã

```
@startuml
```

```
class Document {
    + id: str
    + title: str
    + author: str
    + file_path: str
    + file_size: int
    + extension: str
    + tags: List[str]
    + date_added: float
    + cover_path: str
    + to_dict() : dict
}
```

```
class MetadataNormalizer {
```

```
+ normalize_author(raw_author: str) : str
+ normalize_title(raw_title: str) : str
+ generate_document_id(file_path: str) : str
}
```

Document -- MetadataNormalizer

@enduml

7. Luồng xử lý (Data Flow):

- Đầu vào: Dữ liệu thô từ PDF/EPUB.
- Gọi MetadataNormalizer.normalize_author(author).
- Khởi tạo object Document với dữ liệu đã làm sạch.
- Truyền Document.to_dict() cho TDD-001 để lưu vào SQLite.

8. Interface / Abstract Base Classes: Không. Dùng dataclasses của Python.

9. API nội bộ (Public Methods):

- clean_metadata(raw_dict: dict) -> dict

10. Xử lý lỗi:

- Nếu file không có tiêu đề, tự động lấy tên file (bỏ đuôi mở rộng) làm tiêu đề.
- Nếu không có tác giả, gán là "Unknown".

11. Tiêu chuẩn coding:

- Sử dụng @dataclass để viết code ngắn gọn.

12. Checklist nghiệm thu:

- [] Chuyển đổi chuẩn xác tên tác giả bị ngược.
- [] Hàm băm MD5 tạo ID duy nhất từ đường dẫn file không bao giờ bị trùng.

13. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-002, viết module models.py. Sử dụng @dataclass tạo class Document với các trường id, title, author, file_path, file_size, extension, tags, cover_path. Viết class MetadataNormalizer có các hàm tĩnh (staticmethod) clean_author, clean_title,

generate_id (sử dụng hashlib.md5 băm filepath). Viết code test sinh ra một Document từ chuỗi thô 'Python Cơ Bản' và tên tác giả 'Nam, Nguyễn', in ra console kết quả đã chuẩn hóa."

TDD-003: EPUB & AZW3 Extractor

1. Mục tiêu:

Trích xuất siêu dữ liệu và ảnh bìa từ các định dạng sách điện tử mã nguồn mở/đóng (EPUB, MOBI, AZW3) với tốc độ cao, không làm hỏng file gốc.

2. Phạm vi:

- Phân tích file EPUB bằng cách đọc cấu trúc file ZIP (EPUB bản chất là file ZIP) hoặc dùng thư viện EbookLib.
- Tìm và đọc file content.opf để lấy Metadata chuẩn Dublin Core.
- Trích xuất file ảnh bìa (Cover Image) lưu ra thư mục cache.

3. Trách nhiệm của module:

- Trả về một dictionary chứa Metadata (Title, Author, Publisher).
- Trả về đường dẫn đến ảnh bìa đã được lưu tạm.

4. Kiến trúc:

- Thuộc tầng: Infrastructure Layer.

5. Sơ đồ UML/PlantUML:

Đoạn mã

@startuml

skinparam componentStyle rectangle

component "File Importer" as imp

component "EPUB Extractor" as epub

component "EbookLib / ZipFile" as lib

component "Cache Manager" as cache

imp -- epub

epub -- lib

lib -- cache

@enduml

6. Class Diagram:

Đoạn mã

@startuml

```
class EpubExtractor {  
    - cache_mgr: CoverCacheManager  
    + extract_metadata(file_path: str) : dict  
    + extract_cover(file_path: str, doc_id: str) : str  
    - _parse_opf(content: str) : dict  
}
```

@enduml

7. Luồng xử lý (Data Flow):

- Mở file dạng read-only.
- Nếu là EPUB: Giải nén ảo trên RAM -> Tìm file .opf -> Parse XML -> Lấy tag <dc:title>, <dc:creator>.
- Tìm tag <meta name="cover"> -> Lấy đường dẫn file ảnh trong file nén -> Lưu luồng byte ảnh ra thư mục Cache (TDD-005).

8. Threading model:

- File I/O luôn chạy trên Background Worker.

9. Xử lý lỗi:

- Bắt lỗi zipfile.BadZipFile nếu file EPUB bị lỗi hỏng.
- Trả về dict rỗng thay vì làm văng (crash) app nếu file không chứa metadata.

10. Checklist nghiệm thu:

- [] Trích xuất đúng tên tác giả và ảnh bìa từ file EPUB.
- [] Thời gian trích xuất dưới 0.1 giây / file (vì không load toàn bộ nội dung HTML).

11. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-003, viết module epub_extractor.py. Không dùng thư viện bên thứ 3 ngoài tiêu chuẩn nếu có thể, hoặc dùng ebooklib. Tối ưu nhất là dùng module zipfile tích hợp sẵn của Python: Mở file epub, tìm file có đuôi .opf, dùng xml.etree.ElementTree phân tích các thẻ dc:title, dc:creator. Tìm thẻ meta name='cover', đối chiếu với thẻ item để tìm đường dẫn file ảnh bìa bên trong zip, đọc byte ảnh và lưu ra một thư mục 'temp_covers'. Viết code test nhận vào một đường dẫn file epub giả định và in ra metadata."

TDD-004: PDF Extractor

1. Mục tiêu:

Trích xuất dữ liệu từ các file PDF, bao gồm Metadata nhúng, Mục lục (Table of Contents - ToC) và toàn bộ nội dung văn bản (để nạp vào FTS5 Database tìm kiếm).

2. Phạm vi:

- Tích hợp thư viện PyMuPDF (fitz).
- Bóc tách văn bản (Text extraction).
- Trích xuất ảnh bìa PDF (Trang đầu tiên) để làm Cover.

3. Trách nhiệm của module:

- Lấy thông số PDF: Số trang, DRM (có bị khóa pass không).
- Trích xuất 10 trang đầu (hoặc toàn bộ nếu dung lượng nhỏ) để nạp vào hệ thống Full-text search.

4. Kiến trúc:

- Thuộc tầng: Infrastructure Layer.

5. Sơ đồ UML/PlantUML:

Đoạn mã

@startuml

skinparam componentStyle rectangle

component "File Importer" as imp

component "PDF Extractor" as pdf

```
component "PyMuPDF (fitz)" as fitz
```

```
component "Database FTS" as fts
```

```
imp -- pdf
```

```
pdf -- fitz
```

```
fitz -- fts
```

```
@enduml
```

6. Class Diagram:

Đoạn mã

```
@startuml
```

```
class PdfExtractor {
```

```
- cache_mgr: CoverCacheManager
```

```
+ extract_metadata(file_path: str) : dict
```

```
+ extract_cover(file_path: str, doc_id: str) : str
```

```
+ extract_text(file_path: str, max_pages: int) : str
```

```
}
```

```
@enduml
```

7. Luồng xử lý (Data Flow):

- Gọi fitz.open(path).
- Đọc doc.metadata lấy title, author.
- Hàm extract_cover: Lấy page 0 -> get_pixmap() -> lưu dạng file PNG vào cache.
- Hàm extract_text: Lặp qua các page, gọi get_text(), nối chuỗi. Trả về chuỗi text để đưa vào DB.

8. Xử lý lỗi:

- Bắt lỗi PDF bị mã hóa (Encrypted). Đánh dấu document là "Encrypted" và bỏ qua bước extract text.

9. Checklist nghiệm thu:

- [] Lấy chính xác metadata của file PDF.
- [] Render trang đầu tiên thành ảnh PNG thu nhỏ.

10. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-004, viết module pdf_extractor.py. Sử dụng thư viện PyMuPDF (import fitz). Viết class PdfExtractor với 3 hàm: extract_metadata đọc doc.metadata; extract_cover lấy doc[0].get_pixmap(matrix=fitz.Matrix(0.5, 0.5)) để giảm kích thước và lưu ra file .png; extract_text lặp qua 10 trang đầu dùng get_text() để gom chữ. Có khối try-except bắt lỗi file bị cài password mã hóa. Code test mở một file pdf có thật trên máy, in ra title và text trang đầu."

TDD-005: Cover Art & Cache Manager

1. Mục tiêu:

Quản lý tập trung toàn bộ hình ảnh bìa sách. Sinh ra các file ảnh kích thước nhỏ (Thumbnails) để giao diện hiển thị mượt mà khi người dùng cuộn qua danh sách hàng ngàn cuốn sách.

2. Phạm vi:

- Thay đổi kích thước ảnh (Resize / Compress) dùng Pillow.
- Lưu trữ file ảnh với tên là ID của Document (đảm bảo không trùng lặp).
- Cung cấp đường dẫn tuyệt đối cho UI load ảnh.

3. Trách nhiệm của module:

- Tiếp nhận luồng byte ảnh từ EPUB/PDF Extractor.
- Nén ảnh xuống chuẩn (ví dụ: chiều ngang tối đa 300px, định dạng WebP hoặc JPEG chất lượng 80%).

4. Kiến trúc:

- Thuộc tầng: Infrastructure Layer.

5. Class Diagram:

Đoạn mã

@startuml

```
class CoverCacheManager {
```

```

- cache_dir: str

+ save_cover(doc_id: str, raw_image_bytes: bytes) : str

+ get_cover_path(doc_id: str) : str

+ clear_cache() : void

}

@enduml

```

6. Luồng xử lý:

- Nhận byte array -> Mở bằng Image.open(io.BytesIO(raw_bytes)) -> Tính toán tỷ lệ -> resize() -> save('cache_dir/doc_id.webp') -> Trả về đường dẫn cho Extractor.

7. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-005, viết module cover_manager.py. Sử dụng thư viện Pillow (PIL) và os. Class CoverCacheManager nhận vào thư mục lưu trữ (vd: %APPDATA%/SmartDoc/covers). Hàm save_cover nhận byte ảnh gốc, dùng PIL resize ảnh sao cho chiều rộng tối đa 300px (giữ nguyên tỷ lệ), nén và lưu thành file định dạng .webp. Trả về đường dẫn file đó. Viết code test tạo ảnh màu ngẫu nhiên (hoặc đọc 1 ảnh mẫu), đưa qua cache manager và in ra đường dẫn file kết quả."

TDD-006: File System Watcher

1. Mục tiêu:

Giúp phần mềm luôn đồng bộ với đĩa cứng. Khi người dùng tải một file ebook mới vào thư mục Downloads hoặc My Books, phần mềm tự động phát hiện, lấy Metadata và hiện lên giao diện ngay lập tức mà không cần bấm nút "Quét thủ công".

2. Phạm vi:

- Lắng nghe các sự kiện của hệ điều hành (File Created, Modified, Deleted, Moved).
- Tích hợp thư viện watchdog.

3. Trách nhiệm của module:

- Giám sát các thư mục cấu hình trong Settings (TDD-015).
- Khi có sự kiện -> Bắt một Event (vd: FileCreatedEvent) thông qua Event Bus (TDD-008).

4. Sơ đồ UML/PlantUML:

Đoạn mã

```
@startuml
```

```
skinparam componentStyle rectangle
```

```
component "OS File System" as os
```

```
component "Directory Watcher" as watcher
```

```
component "Event Bus" as bus
```

```
component "Job Queue" as queue
```

```
os -- watcher : file_added
```

```
watcher -- bus : publish(FileDetected)
```

```
bus -- queue : trigger import job
```

```
@enduml
```

5. Luồng xử lý:

- User cấu hình theo dõi thư mục "D:\Ebooks".
- watchdog.Observer chạy nền.
- User copy file book.pdf vào thư mục -> Sự kiện on_created kích hoạt -> Gửi filepath qua EventBus.

6. Xử lý lỗi (Debouncing):

- Khi copy file lớn, OS sẽ bắn sự kiện on_modified liên tục trong khi file đang được ghi. Cần cơ chế Debounce (Chờ file ghi xong hoàn toàn hoặc kích thước không đổi trong 2 giây) mới bắt đầu nạp vào hệ thống, tránh lỗi FileInUse.

7. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-006, viết module file_watcher.py. Sử dụng thư viện watchdog.observers và watchdog.events.FileSystemEventHandler. Xây dựng class LibraryWatcher. Ghi đè các hàm on_created, on_deleted, on_moved. Rất quan trọng: Viết một cơ chế Debounce đơn giản (dùng threading.Timer hoặc lưu timestamp) để tránh việc xử lý file khi nó đang được copy (hệ điều hành chưa nhả lock). Khi file đã sẵn sàng, hãy gọi một callback hoặc in ra console. Viết code test chạy observer theo dõi thư mục hiện tại trong 10 giây."

TDD-007: Background Job Queue (Importer)

1. Mục tiêu:

Điều phối tiến trình nạp file (Import) hàng loạt vào Database mà không làm treo (freeze) giao diện người dùng. Có thể nạp 10,000 file PDF/EPUB cùng lúc ở chế độ nền.

2. Phạm vi:

- Hàng đợi (Queue) chứa đường dẫn các file cần xử lý.
- Một nhóm luồng Worker (Thread Pool) hoặc đa tiến trình (Process Pool) để xử lý file.

3. Trách nhiệm của module:

- Lấy đường dẫn file từ Queue.
- Gọi TDD-003 (EPUB) hoặc TDD-004 (PDF) dựa theo đuôi mở rộng file.
- Chuẩn hóa bằng TDD-002.
- Ghi vào SQLite (TDD-001).
- Báo cáo tiến độ (Progress: x/1000) về giao diện UI.

4. Sơ đồ UML/PlantUML:

Đoạn mã

@startuml

skinparam componentStyle rectangle

component "Queue" as queue

component "Worker Threads" as workers

component "Extractors" as extractors

component "Database" as db

queue -- workers : get_path

workers -- extractors : extract_metadata

extractors -- workers : return Document

workers -- db : insert_document

@enduml

5. Luồng xử lý:

- Thư mục có 100 file. Vòng lặp `os.walk` đẩy 100 đường dẫn vào `queue.Queue`.
- 4 luồng Worker thức dậy, liên tục lấy file từ `Queue`.
- Kết nối độc lập với SQLite (sử dụng WAL mode). Xử lý xong, gửi Signal/Event lên UI cập nhật thanh trạng thái.

6. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-007, viết module `import_job_queue.py`. Sử dụng `queue.Queue` và `threading.Thread`. Class `ImportQueueManager` có hàm `add_file(path)`. Khởi tạo 4 Worker threads (hàm `worker_loop`). Hàm `worker` lấy file từ `queue`, có một vòng logic giả lập: nhận diện đuôi file (pdf/epub), sleep 0.1s giả lập extract, sinh ra mock Document dict, gọi một callback báo thành công. Đảm bảo hỗ trợ Stop Event để thoát luồng gọn gàng khi app tắt. Test đẩy 20 file mock vào `queue` và in ra kết quả đa luồng."

TDD-008: Event Bus System

1. Mục tiêu:

Hệ thống giao tiếp nội bộ "không khớp nối" (Decoupled). Bất kỳ sự thay đổi dữ liệu nào (ví dụ: Database vừa nạp xong 10 sách mới) cũng sẽ được phát sóng ra toàn hệ thống để các module khác (ví dụ: Grid View) tự động cập nhật mà không cần gọi hàm chéo nhau.

2. Phạm vi:

- Hoạt động tương tự Event Bus của PA Studio Pro nhưng được tối ưu hóa cho SmartDoc Library.
- Định nghĩa các luồng sự kiện (Events) lõi như: `LibraryUpdated`, `FileDeleted`, `SearchRequested`.

3. Trách nhiệm của module:

- Cung cấp cơ chế Đăng ký (Subscribe) và Phát (Publish).

4. Luồng xử lý:

- Ví dụ: Omnibar gõ từ khóa -> `EventBus.publish(SearchRequested(query="python"))`
-> Grid View nhận Event -> Gọi Database -> Update lại màn hình.

5. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-008, tái sử dụng và tinh chỉnh module Event Bus. Viết file event_bus.py. Định nghĩa class BaseEvent. Tạo class EventBus chuẩn Singleton hoặc dùng Dependency Injection, hỗ trợ Thread-Safe với threading.Lock. Viết các event class cụ thể cho dự án: LibraryUpdatedEvent, SearchRequestedEvent(query). Viết code test mô phỏng UI đăng ký nhận SearchRequestedEvent và in ra từ khóa."

TDD-009: Smart Rules & Virtual Collections

1. Mục tiêu:

Tạo ra các "Thư mục ảo" (Smart Collections) tự động nhóm các tài liệu lại với nhau dựa trên một bộ quy tắc logic (Rules) thay vì người dùng phải copy file bằng tay.

2. Phạm vi:

- Phân tích cú pháp quy tắc: (VD: Ext == 'pdf' AND Year > 2020 AND Tag contains 'AI').
- Biến đổi các quy tắc này thành câu lệnh SQL của bảng FTS5 hoặc bảng chính (TDD-001).

3. Trách nhiệm của module:

- Lưu các Rule này thành dạng JSON xuống Database (Bảng collections).
- Cung cấp API get_collection_sql(collection_id) để UI sử dụng.

4. Kiến trúc:

- Thuộc tầng: Application Layer.

5. Class Diagram:

Đoạn mã

@startuml

```
class SmartRule {  
    + field: str  
    + operator: str  
    + value: str  
}
```

```
class VirtualCollection {
```

```

+ name: str
+ rules: List[SmartRule]
+ logic: str 'AND/OR'
+ to_sql_where_clause() : str
}
@enduml

```

6. Luồng xử lý:

- User cấu hình Filter UI -> Sinh đối tượng VirtualCollection.
- Hệ thống gọi to_sql_where_clause(). Chuyển đổi: field='extension', operator='=', value='pdf' => extension = 'pdf'.
- Nối chuỗi SQL và đưa cho TDD-001 query.

7. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-009, viết module smart_collections.py. Định nghĩa class SmartRule và VirtualCollection. Viết logic chuyển đổi trong hàm to_sql_where_clause(). Hỗ trợ các operator: eq (=), contains (LIKE '%...%'), gt (>). Xử lý an toàn chống SQL Injection bằng cách chuẩn bị query với các dấu ? và trả về một tuple (sql_string, parameters). Viết code test tạo Collection tên 'Sách AI mới', rule: năm > 2022 AND tags contains 'AI', in ra câu query SQL kết quả."

TDD-010: Main Window & UI Shell

1. Mục tiêu:

Khung giao diện chính của SmartDoc Library, được thiết kế theo phong cách hiện đại, phẳng (Flat Design), tối giản (Minimalist), tối đa hóa diện tích hiển thị nội dung, tương tự như Notion hoặc Finder.

2. Phạm vi:

- Sử dụng PySide6.
- Layout chia làm 3 phần chính (3-Pane Layout):
 - Bên trái: Sidebar (Danh sách Virtual Collections, Tags).
 - Ở trên (Header): Omnibar Search (TDD-011).

- Trung tâm: Danh sách kết quả dạng Grid/List (TDD-013).

3. Trách nhiệm của module:

- Khởi tạo và lắp ráp các Component UI lại với nhau.
- Quản lý trạng thái cửa sổ (Maximize, Resize).

4. Sơ đồ UML/PlantUML:

Đoạn mã

@startuml

skinparam componentStyle rectangle

component "Main Window" as main

component "Sidebar UI" as sidebar

component "Header (Omnibar)" as header

component "Content Area" as content

main *-- header : top

main *-- sidebar : left

main *-- content : center

@enduml

5. Luồng xử lý:

- Khởi tạo các module UI con -> Nhúng vào QSplitter (để user có thể kéo chỉnh độ rộng Sidebar) và QVBoxLayout.

6. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-010, viết module main_window.py bằng PySide6. Khởi tạo QMainWindow kích thước 1200x800. Sử dụng QSplitter chia màn hình chiều ngang làm 2 phần (Sidebar chiếm 20%, Content chiếm 80%). Tại phần Content, dùng QVBoxLayout, trên cùng đặt một ô tìm kiếm (QLineEdit giả làm Omnibar), bên dưới đặt một vùng trống (dành cho Grid). Thiết kế UI hiện đại, xóa bỏ các viền nổi cũ kỹ của Qt bằng StyleSheet (border: none). Chạy test hiển thị MainWindow lên màn hình."

TDD-011: Omnibar Search UI

1. Mục tiêu:

Thanh tìm kiếm "Quyền năng" (Omnibar). Nhập từ khóa để tìm mọi thứ. Hỗ trợ hiển thị gợi ý (Auto-complete) tức thì ngay khi đang gõ.

2. Phạm vi:

- Thanh Input text nổi bật.
- Cửa sổ Pop-up thả xuống (Dropdown List) hiển thị kết quả truy vấn nhanh.
- Phân tích chuỗi tìm kiếm nâng cao (VD: author:Nam text:Python).

3. Trách nhiệm của module:

- Lắng nghe sự kiện textChanged của Qt.
- Gửi Event SearchRequestedEvent (kèm độ trễ Debounce 300ms để không gửi liên tục 100 query khi gõ 10 chữ).

4. Luồng xử lý:

- Gõ chữ "P" -> "Py" -> "Pyt" -> Đợi 300ms (không gõ tiếp) -> QTimer timeout -> Bắn Event SearchRequested("Pyt").
- Nhận kết quả từ DB (thông qua Event Bus) -> Đổ vào list kết quả thả xuống.

5. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-011, viết module omnibar_ui.py bằng PySide6. Class kế thừa QLineEdit. Yêu cầu quan trọng: Tạo cơ chế Debounce sử dụng QTimer (setSingleShot = True, interval = 300ms). Bắt sự kiện textChanged, reset lại timer. Khi timer timeout, phát ra một pyqtSignal(str) chứa từ khóa. Viết code trang trí QLineEdit có bo góc (border-radius), padding lớn, font chữ to. Viết file test kết nối Signal đó với một hàm print in ra màn hình."

TDD-012: Faceted Filter Panel

1. Mục tiêu:

Thanh công cụ bên trái (Sidebar) cho phép thu hẹp kết quả tìm kiếm bằng cách nhấp chọn nhiều khía cạnh cùng lúc (Ví dụ: Tick vào định dạng [x] PDF, chọn năm [2023]).

2. Phạm vi:

- Hiển thị danh sách các Thẻ (Tags), Tác giả (Authors), Định dạng (Formats) được gom nhóm lại với số lượng tương ứng. Ví dụ: Python (120), AI (45).

3. Trách nhiệm của module:

- Truy vấn DB (TDD-001) để lấy danh sách Group By và Count.
- Cập nhật trạng thái các Checkbox.
- Tổng hợp các tick này thành VirtualCollection rules (TDD-009) để lọc thư viện chính.

4. Luồng xử lý:

- DB nạp file mới -> Bắn event LibraryUpdated -> Filter Panel truy vấn DB đếm lại số lượng Tag -> Render lại QTreeWidget hoặc QListWidget.

5. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-012, viết module filter_sidebar.py bằng PySide6. Sử dụng QTreeWidget (chỉ hiển thị 1 cột, ẩn header). Tạo các root item như 'Định dạng', 'Tác giả', 'Tags'. Viết một hàm mock update_filters(data_dict) nhận dict đếm số lượng (vd: {'Định dạng': {'pdf': 10, 'epub': 5}}) và render thành các child item có checkbox (Qt.ItemIsUserCheckable). Bắt sự kiện thay đổi checkbox để in ra danh sách các filter đang được tick."

TDD-013: Grid & List View

1. Mục tiêu:

Vùng hiển thị kết quả chính, cho phép người dùng lướt qua hàng ngàn bìa sách một cách mượt mà và trực quan, hỗ trợ chuyển đổi giữa chế độ Lưới (Grid - hiện nhiều bìa) và Danh sách (List - hiện nhiều metadata).

2. Phạm vi:

- Sử dụng mô hình Model/View chuẩn của Qt (QListView, QAbstractListModel) để quản lý bộ nhớ siêu việt. KHÔNG DÙNG vòng lặp tạo ra hàng ngàn Widget (sẽ làm treo máy).
- Load ảnh bìa không đồng bộ (Asynchronous Image Loading) từ TDD-005.

3. Trách nhiệm của module:

- Render UI dựa trên danh sách Document dict trả về từ Database.
- Tính toán kích thước (Resize) Grid Item linh hoạt khi kéo độ rộng cửa sổ.

4. Luồng xử lý:

- Nhận mảng dữ liệu (VD: 500 records) -> Nạp vào DocumentModel.

- Qt tự động gọi hàm data() chỉ cho những thẻ sách hiển thị trên màn hình hiện tại (khoảng 20 thẻ).
- Trong hàm data(), gọi cache load ảnh bìa và nhả ra cho UI.

5. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-013, viết module grid_view.py bằng PySide6. Đặc biệt yêu cầu dùng kiến trúc Model/View: Tạo class LibraryModel kế thừa QAbstractListModel. Ghi đè hàm rowCount và data. Hàm data sẽ xử lý vai trò Qt.DisplayRole (trả về title sách) và Qt.DecorationRole (trả về QIcon lấy từ file mock). Tạo QListView, đặt setViewMode(QListView.IconMode) và setResizeMode(QListView.Adjust) để các icon tự động flow giống Grid. Test nạp 1000 item giả (chỉ text/icon rỗng) vào Model để kiểm tra tốc độ cuộn."

TDD-014: File Action Engine

1. Mục tiêu:

Cung cấp các lệnh thao tác (Actions) thực tế với file vật lý của tài liệu khi người dùng Click chuột phải hoặc Double Click trên giao diện.

2. Phạm vi:

- Mở file bằng phần mềm mặc định của hệ điều hành (Ví dụ: Double click file PDF sẽ mở bằng Acrobat Reader trên Windows, hoặc Preview trên Mac).
- Tính năng "Open file location" (Mở thư mục chứa file và highlight nó trong File Explorer).
- Xóa file (Chỉ xóa chỉ mục khỏi phần mềm, hoặc xóa cả file gốc dưới đĩa cứng).

3. Trách nhiệm của module:

- Giao tiếp với shell của Hệ điều hành.
- Nhận lệnh thông qua Context Menu (Click chuột phải).

4. Luồng xử lý:

- Chuột phải -> Lấy Document ID đang chọn -> Gọi DB lấy file_path.
- Lệnh Mở: Windows dùng os.startfile(path), macOS dùng subprocess.run(['open', path]).
- Lệnh Mở thư mục: Dùng subprocess.Popen(r'explorer /select,{path}').

5. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-014, viết module file_actions.py. Import os, sys, subprocess. Định nghĩa class tĩnh FileActionEngine. Viết hàm đa nền tảng (if sys.platform == 'win32'): open_file(filepath) sử dụng os.startfile, và show_in_file_manager(filepath) sử dụng subprocess gọi explorer.exe /select. Viết hàm delete_file(filepath) xử lý exception kỹ lưỡng (FileNotFound). Code độc lập test gọi hàm show_in_file_manager trở vào file code python hiện tại."

TDD-015: App Configuration

1. Mục tiêu:

Quản lý các thiết lập riêng của người dùng cho dự án SmartDoc Library.

2. Phạm vi:

- Lưu trữ danh sách các Thư mục (Watch Folders) mà phần mềm cần theo dõi (Ví dụ: C:\Downloads, D:\Ebooks).
- Lưu tùy chọn hiển thị: Dark/Light Mode, Grid/List view mode.
- Đọc/Ghi ra file settings.json trong %APPDATA%\SmartDocLibrary.

3. Kiến trúc:

- Kế thừa mô hình TDD-007 của PA Studio Pro, nhưng cấu trúc key-value sẽ chuyên biệt cho dự án này.

4. Luồng xử lý:

- Lúc khởi động -> Load file -> Trả danh sách folders cho FileSystemWatcher (TDD-006).

5. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-015, tái tạo module quản lý cấu hình. Viết class ConfigManager. Định nghĩa cấu trúc DEFAULT_CONFIG gồm danh sách 'watch_folders', boolean 'dark_mode', string 'view_mode'. Xử lý đọc ghi JSON an toàn. Nếu file không tồn tại, tự sinh file với default. Viết code test thêm một đường dẫn thư mục vào danh sách 'watch_folders', gọi save(), đọc lại và assert."

TDD-016: Cloud Review System (Google Drive Sync)

1. Mục tiêu: Hệ thống cho phép người dùng ẩn danh (chỉ cần Nickname) để lại đánh giá 5 sao và nhận xét cho từng cuốn sách. Dữ liệu được đồng bộ hóa tập trung qua một tài khoản Google Drive cấu hình sẵn (không cần server backend).

2. Phạm vi:

- Sử dụng thư viện google-api-python-client và google-auth kết nối qua Service Account.
- Lưu trữ trên Drive: Mỗi cuốn sách có 1 file tên <document_id>_reviews.json.
- Giao diện (UI) thiết kế dạng "Collapsible / Accordion" (Bấm vào mới xổ ra) để tiết kiệm diện tích.

3. Trách nhiệm của module:

- Đọc/Lưu Nickname vào ConfigManager (TDD-015).
- Kéo (Download) file review từ Drive về, gộp với review mới của người dùng, và Đẩy (Upload/Update) lại lên Drive.

4. Kiến trúc & Data Flow:

- Người dùng bấm "Gửi nhận xét" -> Tạo object JSON {"nickname": "Kenvin", "rating": 5, "comment": "Sách hay", "timestamp": 1690000000}.
- Gọi API Google Drive tìm file <id>.json. Nếu có -> Download -> Append object -> Update. Nếu chưa -> Create mới.
- Quá trình I/O mạng phải chạy trên Background Thread để không đơ giao diện.

5. Sơ đồ UML/PlantUML:

Đoạn mã

@startuml

skinparam componentStyle rectangle

component "Review UI" as ui

component "DriveManager" as drive

cloud "Google Drive" as cloud

ui -> drive : submit_review(doc_id, rating, comment)

drive -> cloud : download_existing_json

drive -> drive : merge_data

drive -> cloud : upload_json

drive -> ui : emit(SyncCompleted)

@enduml

6. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-016, viết 2 class trong file cloud_reviews.py. Phần 1: GoogleDriveSync, sử dụng google-api-python-client (xác thực bằng file service_account.json truyền vào). Viết hàm sync_review(doc_id, nickname, rating, text) có logic tìm file theo tên trên Drive, tải về, nối thêm dictionary review, sau đó ghi đè lên Drive. Phần 2: Viết giao diện ReviewPanel bằng PySide6. Có 1 nút QPushButton làm tiêu đề, khi click sẽ thay đổi setVisible() của một vùng QWidget chứa danh sách review (tạo hiệu ứng xổ xuống - Accordion). Yêu cầu chạy luồng mạng ngầm."

TDD-017: AI Summarizer Engine

1. Mục tiêu: Đọc nội dung tài liệu và sinh ra một bản tóm tắt ngắn gọn, súc tích (Abstract) giúp người dùng nắm bắt nội dung sách mà không cần đọc hết.

2. Phạm vi:

- Trích xuất 10,000 ký tự đầu tiên của cuốn sách (thông qua Extractor TDD-003, TDD-004) làm dữ liệu đầu vào.
- Gửi Prompt tới API của Google Gemini (hoặc OpenAI) để tóm tắt.
- Lưu trữ vĩnh viễn kết quả tóm tắt vào bảng documents của SQLite để tái sử dụng và phục vụ tìm kiếm ngữ nghĩa sau này.

3. Trách nhiệm của module:

- Cung cấp nút bấm "Tạo Tóm tắt AI" trên giao diện chi tiết sách.
- Khi người dùng click: Hiện loading spinner -> Trích xuất text -> Gọi API AI -> Lưu DB -> Render kết quả ra màn hình.

4. Luồng xử lý (Data Flow):

- Prompt chuẩn hóa: *"Dưới đây là một phần nội dung của cuốn sách {title}. Hãy viết một tóm tắt nội dung chính khoảng 150 từ, phân tích thể loại và đối tượng độc giả phù hợp: \n {text}"*

- Nhận phản hồi -> Lưu vào cột ai_summary của SQLite -> Bắn Event DocumentUpdated.

5. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-017, viết module ai_summarizer.py. Sử dụng thư viện google-generativeai. Xây dựng class AISummarizerEngine có hàm generate_summary(title, raw_text, api_key). Viết cấu trúc Prompt hướng dẫn AI tóm tắt nội dung và xác định thể loại. Đảm bảo API call được bọc trong try-except và chạy trong QRunnable hoặc QThread của PySide6. Viết thêm một Widget giao diện có nút 'Tạo tóm tắt', khi bấm sẽ hiện 'Đang xử lý...' và cập nhật QTextBrowser khi có kết quả trả về từ Signal."

TDD-018: Semantic Search Engine (Tìm kiếm ngữ nghĩa)

1. Mục tiêu: Đây là tính năng "Sát thủ". Cho phép tìm kiếm bằng ngôn ngữ tự nhiên, dựa trên ý nghĩa (Semantic) thay vì chỉ khớp từ khóa. (Ví dụ: "*sách ngôn tình hiện đại trung quốc đọc trong 5 ngày*").

2. Phạm vi:

- Không dùng FTS5 cho tính năng này. Sử dụng thư viện **sentence-transformers** để tạo Vector Embeddings (mảng số thực đại diện cho ý nghĩa).
- Sử dụng mô hình nhẹ, đa ngôn ngữ hỗ trợ tiếng Việt tốt (Ví dụ: paraphrase-multilingual-MiniLM-L12-v2).
- Chạy 100% cục bộ (Local), không tốn phí API.

3. Trách nhiệm của module:

- Khi thêm sách mới -> Lấy Title + Tags + AI Summary -> Chạy qua mô hình AI biến thành 1 vector kích thước 384 chiều -> Lưu mảng numpy này ra file nội bộ hoặc database.
- Khi gõ câu tìm kiếm -> Biến câu đó thành vector -> Dùng phép toán Cosine Similarity (độ tương đồng cosine) để so sánh vector câu hỏi với tất cả vector của sách.
- Sắp xếp kết quả theo điểm tương đồng cao nhất (Score > 0.6) và trả về.

4. Xử lý Logic thời gian đọc ("**Đọc trong chuyến du lịch 5 ngày**"):

- Thay vì để mô hình tự suy đoán mơ hồ, khi tính điểm hiển thị, module sẽ kết hợp với thuộc tính page_count hoặc file_size (Sách 5 ngày = sách có độ dài trung bình).

5. Sơ đồ UML/PlantUML:

Đoạn mã

```
@startuml
```

```
skinparam componentStyle rectangle
```

```
component "Natural Query" as query
```

```
component "Sentence Transformer" as st
```

```
database "Vector Index (Numpy)" as vec
```

```
component "FTS5 DB" as db
```

```
query -> st : "Ngôn tình hiện đại..."
```

```
st -> vec : encode to Vector(384)
```

```
vec -> vec : Cosine Similarity
```

```
vec -> db : return Top 10 IDs
```

```
@enduml
```

6. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-018, viết module semantic_search.py. Yêu cầu dùng thư viện sentence-transformers và numpy (không cần cài ChromaDB hay FAISS để giữ ứng dụng nhẹ). Viết class SemanticSearchEngine. Hàm init sẽ load mô hình sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2. Viết hàm add_document_to_index(doc_id, text_to_embed) để tạo vector và lưu vào một dictionary {doc_id: vector}. Viết hàm search(query_string, top_k=5). Hàm search sẽ mã hóa query_string, dùng numpy.dot chia cho norm (tính cosine similarity) với toàn bộ vector trong dictionary, sắp xếp lấy top_k ID có điểm cao nhất. Code có mock data test để chứng minh tìm được sách."

Dưới đây là **TDD-017 (Phiên bản cập nhật)** và **TDD-019 (Module mới)** được thiết kế chuẩn xác theo yêu cầu của bạn. Bạn hãy copy toàn bộ phần dưới đây và giao tiếp cho Agent 2 nhé!

Bối cảnh cho Agent 2:

Tôi là Tech Lead của dự án SmartDoc Library. Hãy cập nhật lại logic của module AI (TDD-017) và phát triển thêm một tính năng mới (TDD-019) cho phép người dùng upload sách lên Cloud cá nhân của họ. Hãy đọc kỹ đặc tả và sinh mã Python.

TDD-017: AI Book Review & Preview Engine (Cập nhật)

1. Mục tiêu:

Phân tích nội dung trích xuất từ tài liệu và sinh ra một bài "Review/Teaser" (Đánh giá/Giới thiệu). Mục đích **KHÔNG PHẢI** là tiết lộ hết nội dung (Spoil), mà là để người đọc nắm bắt được văn phong, không khí (vibe) của câu chuyện, chủ đề chính, và đánh giá xem cuốn sách này phù hợp với đối tượng nào.

2. Phạm vi:

- Trích xuất khoảng 10,000 ký tự đầu tiên hoặc phần giới thiệu của cuốn sách.
- Gửi Prompt tới API (Google Gemini / OpenAI).
- Lưu kết quả review vào Database để sử dụng cho tính năng Tìm kiếm Ngữ nghĩa (TDD-018) và hiển thị trên giao diện.

3. Trách nhiệm của module:

- Thiết kế một cấu trúc Prompt chuyên nghiệp đóng vai trò như một nhà phê bình sách.
- Xử lý bất đồng bộ để UI không bị treo khi chờ AI phản hồi.

4. Cấu trúc Prompt tiêu chuẩn:

- *"Bạn là một nhà phê bình sách chuyên nghiệp. Dưới đây là phần mở đầu của cuốn sách {title}. Hãy viết một bài review ngắn gọn (khoảng 150-200 từ) bao gồm: 1. Câu hook tóm tắt sự hấp dẫn của chủ đề/cốt truyện. 2. Nhận xét về văn phong và nhịp điệu (hài hước, u ám, học thuật...). 3. Khuyến nghị cuốn sách này dành cho đối tượng độc giả nào. Tuyệt đối không tiết lộ trước kết quả hay nội dung quan trọng."*

5. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-017 (Cập nhật), viết module ai_book_reviewer.py. Class AIBookReviewer có hàm generate_preview(title, raw_text, api_key). Bắt buộc sử dụng cấu trúc Prompt định hướng AI đóng vai trò nhà phê bình sách như đặc tả. Chạy gọi API trong môi trường Background (sử dụng QThread hoặc ThreadPoolExecutor). Hàm trả về text markdown. Viết đoạn test giả lập 1 đoạn văn bản ngẫu nhiên, gọi mock AI và in ra console kết quả bài review."

TDD-019: Personal Cloud Exporter (Send to Cloud)

1. Mục tiêu:

Cho phép người dùng xuất (Upload) trực tiếp file sách gốc (.epub, .pdf, .pdf) từ phần mềm SmartDoc Library lên tài khoản Google Drive cá nhân của họ để dễ dàng đọc trên các thiết bị khác.

2. Phạm vi:

- Xác thực OAuth2 (User Login qua trình duyệt) với API Google Drive.
- Lưu trữ token truy cập (Access Token) cục bộ để người dùng không phải đăng nhập lại mỗi lần gửi sách.
- Cho phép người dùng cấu hình đường dẫn thư mục đích trên Drive (Ví dụ: Tai_Lieu/Sach_Du_Lich).
- Tự động tạo thư mục gốc tên là ebook trên Drive nếu người dùng không cung cấp đường dẫn.

3. Trách nhiệm của module:

- Mở luồng xác thực trình duyệt (OAuth Flow).
- Phân tích đường dẫn người dùng nhập vào để tìm kiếm hoặc tạo ID thư mục tương ứng trên Google Drive.
- Tải file vật lý lên Drive, có báo cáo % tiến trình (Progress).

4. Kiến trúc & Data Flow:

- Người dùng chọn "Send to Cloud" trên UI -> Nhập path (vd: để trống).
- Hệ thống check file token.json ở máy tính. Nếu chưa có -> Bật trình duyệt yêu cầu cấp quyền Google Drive -> Lưu file token.
- Hệ thống gọi API Drive tìm xem thư mục tên ebook có tồn tại ở thư mục gốc (root) không.
- Nếu không có -> Tạo thư mục ebook -> Lấy ID.
- Dùng MediaFileUpload nạp file sách vào thư mục ID đó.
- Thông báo "Upload thành công" qua EventBus/Toast.

5. Sơ đồ UML/PlantUML:

Đoạn mã

@startuml

```
skinparam componentStyle rectangle
component "UI / Context Menu" as ui
component "CloudExporter" as exporter
component "OAuth2 Agent" as oauth
cloud "User's Google Drive" as drive
```

```
ui -> exporter : upload(file_path, target_folder)
exporter -> oauth : get_or_refresh_token()
oauth -> drive : authenticate
exporter -> drive : check/create folder
exporter -> drive : upload file (MediaFileUpload)
exporter -> ui : emit Progress/Success
@enduml
```

6. Xử lý lỗi:

- Hết hạn Token -> Tự động dùng Refresh Token làm mới.
- Mất kết nối mạng giữa chừng -> Báo lỗi mạng cho người dùng.
- File đang bị phần mềm khác khóa -> Báo lỗi I/O.

7. Prompt tối ưu để Gemini sinh mã:

"Dựa trên TDD-019, viết module cloud_exporter.py. Yêu cầu sử dụng thư viện google-auth-oauthlib và google-api-python-client. Xây dựng class GoogleDriveExporter. Viết hàm authenticate() sử dụng InstalledAppFlow đọc file credentials.json (do dev cấp) để user login, sau đó lưu token.json để dùng lại. Viết hàm _get_or_create_folder(folder_path): nếu folder_path rỗng thì mặc định tìm/tạo thư mục 'ebook' ở thư mục gốc (root). Viết hàm upload_file(local_path, target_folder_path) sử dụng MediaFileUpload(resumable=True). Đảm bảo code chạy trên luồng phụ và an toàn."